实验七　OpenDaylight 控制器编程实践

一、　实验目的

1. 安装配置 OpenDaylight 控制器编程环境；
2. 基于 OpenDaylight 控制器源码完成网络模块开发；
3. 验证功能模块的运行；
4.程序调试与结果分析；

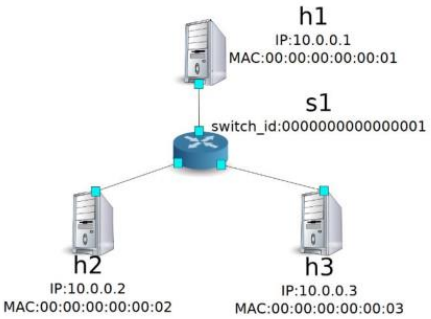
二、　基础实验

1.内容与任务

完成下列实验：

（1）安装、配置和运行 OpenDlight 的 Lithium 版本和 mininet，搭建线性网络拓扑。观察 L2switch 的地址学习过程，运行 postman，用 restapi 记录学习到的地址。

（2）用mininet搭建如下图的 3 个节点的星形拓扑网络，用 YANGGUI 管理交换机的流表，通过流表的增、删、改功能，实现 h1 ping h3 通讯的阻断与恢复。



（3）安装与配置 Postman，通过向 Opendaylight 控制器发送 RESTAPI 请求，实现 h1 ping h3 通讯的阻断与恢复。

三、　实验配置与操作要点

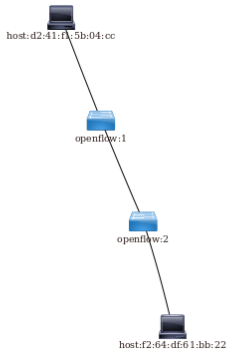
1.　从教学 FTP 中下载 lithium 版本 opendaylight 控制器（文件名 distribution-karaf-0.3.4-Lithium-SR4.tar，或在 sdn 虚拟机中访问网址：https://nexus.opendaylight.org/content/repositories/public/org/opendaylight/integration/distribution-karaf/0.3.4-Lithium-SR4/　下载），下载后在虚拟机中用 tar -zxvf 命令解压。

2.　用 Sudo ./bin/karaf
启动 opendlight，按顺序安装以下特性（各组件的功能可以查看网页 https://docs.opendaylight.org/en/stable-carbon/getting-started-guide/installing_opendaylight.html 中的特性介绍表）：

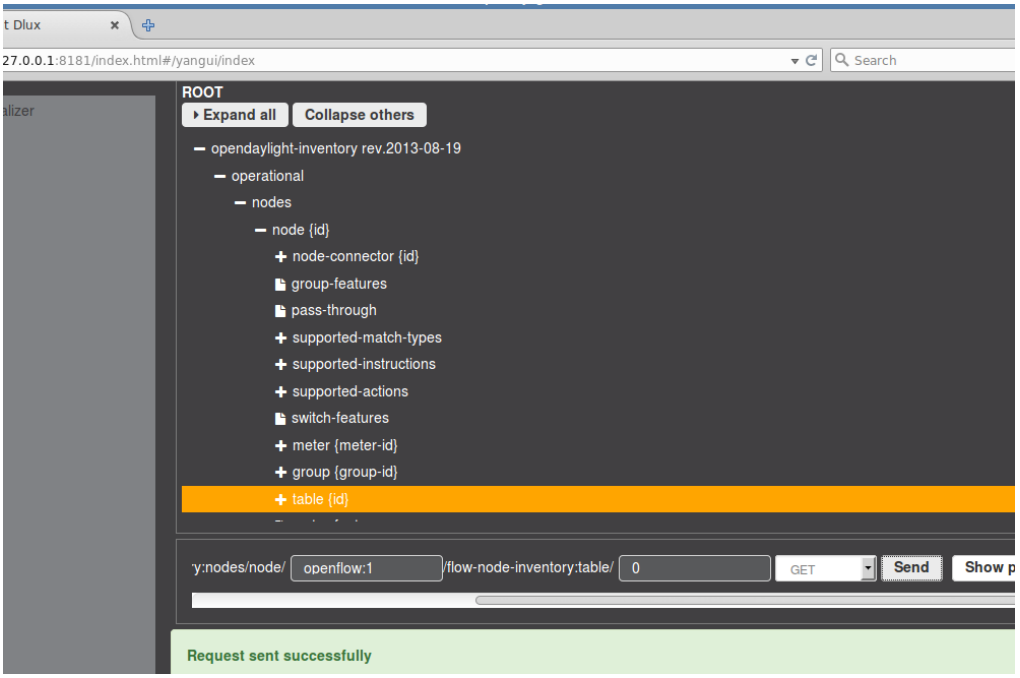
```
feature:install odl-restconf
feature:install odl-l2switch-switch-ui
feature:install odl-openflowplugin-flow-services-ui
```

feature:install odl-mdsal-apidocs
feature:install odl-dlux-all
可以在特性界面输入开头字母后按 tab 键查看输入选择。
输入网页访问，默认用户名密码为:admin/admin
http://127.0.0.1:8181/index.html
查看 topology 页面，并截图记录。

3.使用 mininet 构造如下图的线性网络



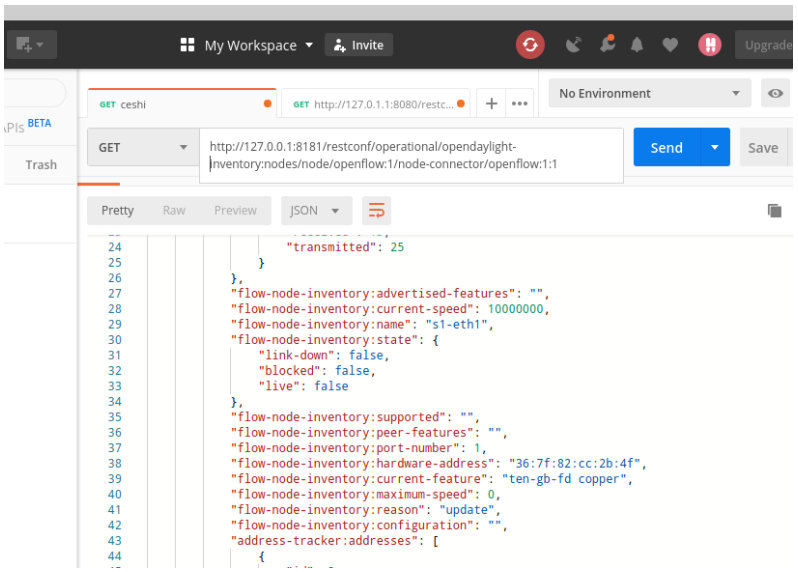
执行：
sudo mn --controller=remote,ip=127.0.0.1 --topo=linear,2 --switch ovsk,protocols=OpenFlow13
查看 topology 页面，点击 reload 按钮，并截图记录。
4.观察地址学习过程，首先用 opendlight 的 web 页面的 YANGUI，查看获取流表的 RESTAPI 写法，如图：



在实验报告中截图记录返回的请求结果：

然后在 Postman 里测试该 RESTAPI（发送请求时，认证方法选 basic auth，用户名和密码使用 admin），记录返回的数据：

随后在 Postman 里，测试获取连接的 RESTAPI
http://127.0.0.1:8080/restconf/operational/opendaylight-inventory:nodes/node/openflow:1/node-connector/openflow:1:1
获得结果类似下图：

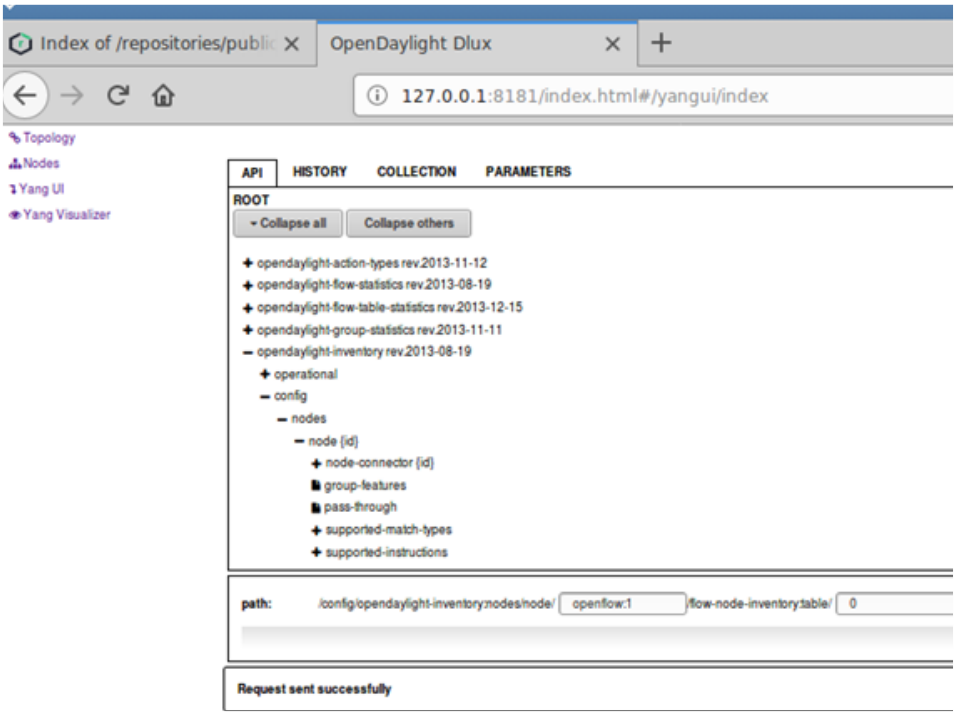


然后在 mininet 中执行 pingall，再次发送该 restapi 并记录学习到的 MAC 地址，截图记入实验报告。

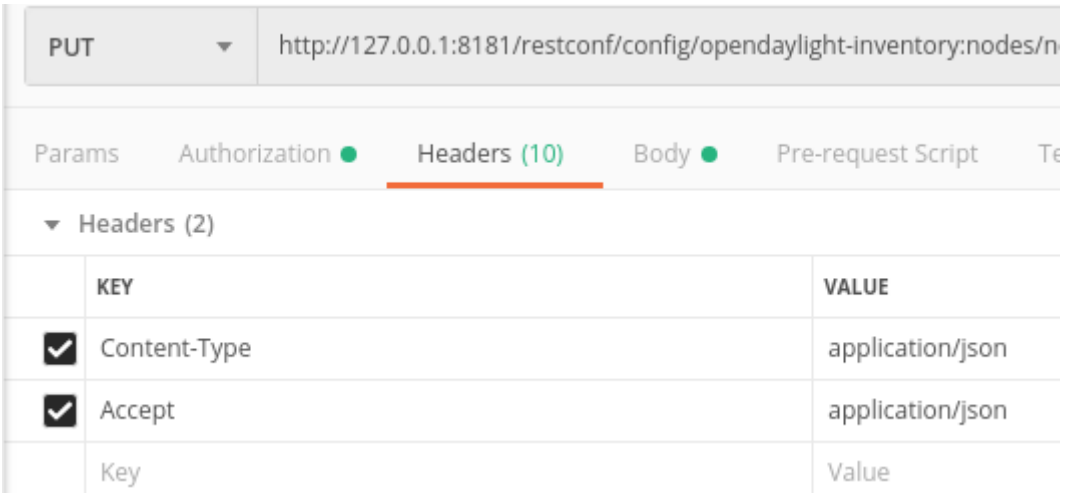
测试如下的拓扑结构中的主机 RESTAPI 请求结果
http://127.0.1.1:8080/restconf/operational/network-topology:network-topology/topology/flow:1
并截图记录，分析返回数据结构中的节点地址和链接信息。

5.关闭前述步骤中的 mininet 和 opendlight，清理 mininet 残留，关闭后重新启动 opendlight。输入
sudo mn --topo=single,3 --mac --controller=remote,ip=127.0.0.1 --switch=ovsk,protocols=OpenFlow13
搭建拓扑，执行 pingall 后在 opendlight 中观察拓扑并截图记录。

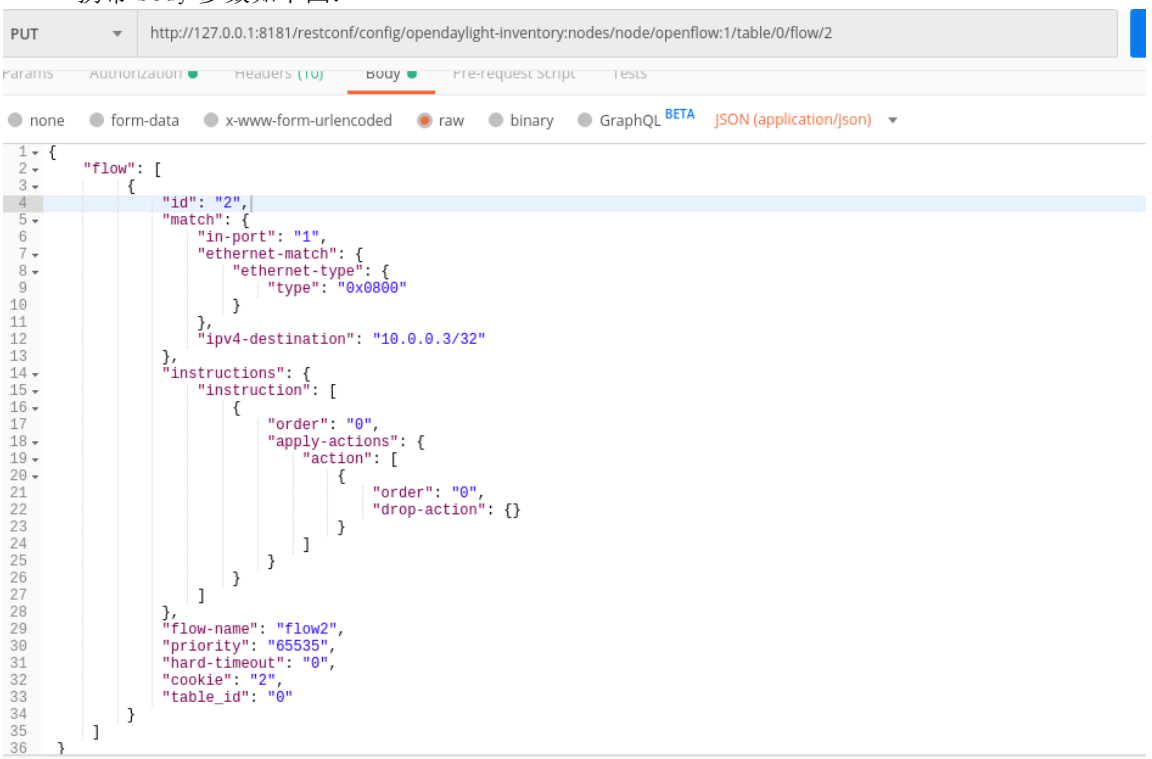
6.在 mininet 中执行
sh ovs-ofctl dump-flows s1 -O OpenFlow13
观察初始流表，然后在 opendlight 的 YangUI 中查看流表



通过 Postman 和 YANGUI 使用 RESTAPI 分别下发流表，阻断 h1 到 h3 之间的通讯，YANGUI 方式查询页面指示，Postman 方式如下：



携带 body 参数如下图：



执行成功后（返回状态为 200 OK），观察新的流表并截图记录，然后执行 h1 ping h3，观察是否被阻断，并截图记录。

注意到上述 RESTAPI 的 URI 写法中使用了 config 而不是前面的 operational，请查询并说明两者的不同，写入实验报告。

8. 在 postman 中执行如图的删除流表 RESTAPI，URI 如下：
http://127.0.0.1:8181/restconf/config/opendaylight-inventory:nodes/node/openflow:1/table/0
观察是否可以再次被 ping 通。然后截图记录。

四、进阶技能实验

1. 在基础实验步骤 2 的星型拓扑中，使用 opendlight 的 RESTAPI，用 Python 编程实现基础技能实验中的 h1 ping h3 通讯的阻断与恢复、以及显示流表的三种操作。

（提示：使用 windows 实体机中的 pycharm 进行 python 开发，开发前虚拟机使用 NAT 模式，禁用实体机的实体网卡，启用 NAT 模式接入的网卡进行开发。在 pycharm 中需要安装 requests 包，用于发送 RESTAPI 请求）

2. 在进阶技能步骤 1 拓扑的基础上，编程完成 Python GUI 网络应用程序，实现防火墙功能。程序通过读入 firewall-policies.csv 规则文件，通过 Opendaylight 控制器向 Openflow 交换机下发对应的流表。对拓扑中的主机进行源和目的 IP 地址匹配，对匹配成功的数据包进行丢弃。